

Теоретическая информатика, осень 2020 г.
Лекция 1. Машина с произвольным доступом к памяти,
псевдокод. Быстрое умножение, метод «разделяй и
властвуй». Сортировка вставкой, сортировка слиянием*

Александр Охотин

6 сентября 2021 г.

Содержание

1 Введение в ТИ	1
2 Модели вычисления	2
3 Быстрое умножение, алгоритм Карацубы	7
4 Алгоритмы сортировки	8

1 Введение в ТИ

Во всякой области технологии — да и вообще едва ли не в каждой области человеческого знания — по мере её развития проявляется некоторое *математическое содержание*, и математические исследования этого содержания определяют развитие технологии. Например, сперва люди строят катапульты или пушки и хотят научиться поточнее из них попадать по неприятелю. Тогда, чтобы рассчитывать траекторию, приходится додуматься до уравнений механики и до математики, нужной для их решения. Математики развивают абстрактную теорию, с её помощью становится возможным разработать более точные модели — а там удаётся построить более совершенные устройства, дальнейшее улучшение которых требует более сложных математических теорий. . . Так технология развивается, а вместе с нею развиваются и научные знания.

В технологиях обработки информации поначалу никакой особенной математики не было, и ещё не так давно для вычислений использовались счёты, а для хранения информации — картотека. Из такого младенческого состояния не вырасти одними инженерными решениями: счёты или картотеку можно понемногу улучшать, можно додуматься даже до арифмометра — но подобными улучшениями не изобрести таких вещей, как компьютер и языки программирования. Прежде должны были возникнуть совершенно новые математические идеи — и они возникли в трудах логиков первой половины XX века, таких, как Курт Гёдель, Стивен Клини, Эмиль Пост, Алан Тьюринг, Алонзо Чёрч. И уже исходя из

*Краткое содержание лекций, прочитанных студентам 1-го курса факультета МКН СПбГУ в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

этих идей появилось представление о том, как соединить электронные элементы, чтобы получился компьютер.

Теоретическая информатика — это, с одной стороны, совокупность математических идей, лежащих в основе технологий обработки информации. Изучая и развивая эти идеи, можно понять существующие технологии и создать новые. С другой стороны, это область чистой математики, вдохновлённая задачами обработки информации; она нужна всем математикам в качестве одного из типов математического мышления, который может оказаться полезным при решении задач в разных областях математики.

1.1 Алгоритмы

Алгоритм — это формальное описание последовательности действий некоторого исполнителя, ведущей к достижению определённой цели. В информатике в качестве исполнителя обычно выступает вычислительное устройство, проверяющее некоторое свойство данного ему на входе объекта, или вычисляющее значение некоторой функции на данном на входе объекте. Например, алгоритм может проверять простоту данного на вход числа, или же вычислять произведение двух данных на входе целых чисел.

С точки зрения программиста, алгоритм — это идея программы, и чтобы научиться писать качественные программы, нужно изучить то, как люди уже умеют это делать — классические алгоритмы. С точки зрения математика, алгоритм — это своего рода формула, позволяющая что-то вычислить, и допускающая удобное математическое доказательство правильности. Алгоритм, работающий быстрее, чем по определению, быстрее, чем интуитивно возможно — это очень интересный математический факт.

2 Модели вычисления

Поскольку алгоритм — это математический объект, для которого предполагается что-то доказывать, это понятие нуждается в формализации.

Машина Тьюринга (Тьюринг [1937]) — математическая модель вычислимости, удобная для доказательств того, что какие-то задачи можно решить алгоритмически, а какие-то нельзя. Настоящие языки программирования для этой цели непригодны, поскольку они слишком сложны: действительно, всякое доказательство, говорящее о *любой программе на C++*, будет вынуждено включать в себя даже не стандарт языка C++, а целиком компилятор, запрограммированный в виде логических умозаключений — в то время как для машины Тьюринга её полное определение занимает всего пол-страницы.

С другой стороны, машина Тьюринга работает не совсем так, как работают реальные компьютеры, время её работы существенно превосходит время работы привычных нам программ, и потому представление алгоритма ею не поможет понять этот алгоритм. Поэтому при описании алгоритмов используется модель высокого уровня — *псевдокод* — условный язык программирования, адаптируемый к каждой конкретной задаче для удобства её изложения. Также в теоретической информатике в качестве более реалистичной разновидности машины Тьюринга используется модель низкого уровня: *машина с произвольным доступом к памяти* — это машинный код условного процессора.

2.1 Псевдокод и его выполнение

Псевдокод пишется на идеализированной модели языка программирования высокого уровня; это программа на условном языке программирования, интуитивно понятном всем, кому доводилось писать программы на каком угодно настоящем языке. Псевдокод легко переводится в программу на любом конкретном языке.

В каждый момент выполняется какая-то одна строчка, а по завершению её выполнения выполняется следующая. Порядок выполнения строчек может быть изменён управляющими конструкциями: условными операторами и операторами цикла. В памяти хранятся все определённые в программе переменные. Пример: вычисление факториала, алгоритм 1.

Пример 1 (Вычисление факториала итерацией).

Процедура $f(n)$

```
1:  $x = 1$ 
2: for  $i = 1$  to  $n$  do
3:      $x = x \cdot i$ 
4: return  $x$ 
```

В каждый момент времени выполняется какая-то строчка в какой-то процедуре, и компьютер держит под рукой все переменные, определённые в этой процедуре (как x в алгоритме 1). Когда эта процедура вызывает какую-то процедуру (неважно, другую или самоё себя), её выполнение приостанавливается, её переменные остаются лежать, где лежат, и начинается выполнение другой процедуры. У той свои переменные, среди которых её параметры; по завершению её выполнения занятая её переменными память освобождается и возобновляется работа той процедуры, которая в своё время её вызвала — её переменные всё это время лежали на своём месте.

Факториал через рекурсию: алгоритм 2.

Пример 2 (Вычисление факториала рекурсией).

Процедура $f(n)$

```
1: if  $n \leq 1$  then
2:     return 1
3: else
4:     return  $n \cdot f(n - 1)$ 
```

Как выполняется псевдокод?

Как это реализуется? Переменные каждого экземпляра вызываемой процедуры размещаются в *стеке*, а рядом с ними записано, из какой строчки программы эту процедуру вызвали (адрес возврата). Поэтому по завершению работы процедуры достаточно переместить указатель стека, освобождая память, занимаемую её переменными, и перейти по адресу возврата.

Дерево вызовов строится для всего выполнения программы. Это корневое дерево, в котором дуги, исходящие из каждой вершины, упорядочены. Каждый вызов процедуры — вершина. Потомки данной вершины — все вызовы, которые делает данная процедура во время работы, перечисленные в том порядке, в котором они вызываются. Путь из корня в некоторую вершину соответствует содержимому стека во время работы данной процедуры.

Пример 3 (Крайне неэффективное вычисление чисел Фибоначчи рекурсией).

Процедура $F(n)$

```
1: if  $n \leq 1$  then
2:     return  $n$ 
```

```
3: else
4:     return  $F(n - 2) + F(n - 1)$ 
```

2.2 Машина с произвольным доступом к памяти

Машина с произвольным доступом к памяти (random access machine, RAM): идеализированная модель компьютера и его машинного кода.

Память: счётное множество ячеек x_i , адресуемых целыми числами ($i \in \mathbb{Z}$). В каждой ячейке в каждый момент времени хранится целое число ($x_i \in \mathbb{Z}$).

Программа состоит из *команд*, пронумерованных, начиная с 1. Команды могут ссылаться на переменные и константы, разрешены следующие способы адресации: n — константа (только для чтения); x_n — ячейка памяти под номером n ; x_{x_n} — ячейка памяти, адрес которой записан в ячейке памяти под номером n (косвенная адресация). Можно разрешить, например, следующие виды команд.

- Пересылка данных: $X = Y$; вычисление арифметического действия: сложение $X = Y + Z$, вычитание $X = Y - Z$, умножение $X = Y * Z$, деление $X = Y / Z$.

После выполнения любой из этих команд, вслед за ней выполняется следующая по номеру команда.

- Безусловный переход: GOTO n , где n — номер команды: следующей выполняется команда с данным номером.
- Косвенный переход: GOTO x_n , номер команды берётся из ячейки памяти с номером n .
- Условный переход: IF $X < Y$ GOTO n , а также с условиями $X == Y$, $X \neq Y$, и т.д.;
- Остановка: HALT.

Пусть входные данные подаются так: в ячейке x_0 записано количество входных значений, $x_0 = n$, а далее в ячейках $x_1, \dots, x_n$ — собственно, сами входные значения. Если машина останавливается, то она оставляет в памяти выходные значения в таком же формате.

Алгоритм 1 Программа для RAM: вычисление суммы чисел

```
1:  $x_{-1} = 1$ 
2:  $x_{-2} = 0$ 
3: IF  $x_{-1} > x_0$  GOTO 7
4:  $x_{-2} = x_{-2} + x_{x_{-1}}$ 
5:  $x_{-1} = x_{-1} + 1$ 
6: GOTO 3
7:  $x_0 = 1$ 
8:  $x_1 = x_{-2}$ 
9: HALT
```

2.3 Компиляция псевдокода

Псевдокод можно перевести в машину с произвольным доступом к памяти — подобно тому, как настоящие языки программирования компилируются в машинный код настоящих компьютеров.

Типы данных: целое число (int), указатель (int *), указатель на указатель (int **), и т.д. Массив: $a[0..n-1]$, реализуется через указатели.

Стек. *Указатель стека* будет храниться в какой-то произвольно выбранной ячейке — скажем, x_{-1} . Функции. Объявление переменных.

При вызове функции в стек записываются её аргументы и адрес возврата (номер команды, которая должна выполняться следующей после окончания работы функции). Вызванная функция при запуске выделяет место в стеке под свои внутренние переменные. Поэтому у каждого экземпляра вызванной функции — свои собственные переменные. При возврате функция освобождает стек и размещает в нём возвращаемое значение.

Условные операторы и операторы цикла реализуются через команды условных переходов.

Алгоритм 2 Более подробный псевдокод для алгоритма 2

Процедура $f(n)$

```

1: if  $n \leq 1$  then
2:   return 1
3: else
4:    $x = n - 1$ 
5:   вызвать  $f(x)$ 
6:   записать вычисленное значение в  $y$ 
7:    $z = n \cdot y$ 
8:   return  $z$ 

```

Основная процедура

```

9:  $x = f(3)$ 
10: return  $x$ 

```

стр.	содержимое стека	действия
1	3 10	перейти к ветви «иначе», выделить место под три переменных
4	0 0 0 3 10	взять n из стека, вычесть единицу, изменить x в стеке
5	2 0 0 3 10	вызов функции f : аргумент x и адрес возврата — в стек
1	2 6 2 0 0 3 10	перейти к ветви «иначе», выделить место под три переменных
4	0 0 0 2 6 2 0 0 3 10	взять n из стека, вычесть единицу, изменить x в стеке
5	1 0 0 2 6 2 0 0 3 10	вызов функции f : аргумент x и адрес возврата — в стек
1	1 6 1 0 0 2 6 2 0 0 3 10	условие $n \leq 1$ выполнено
2	1 6 1 0 0 2 6 2 0 0 3 10	оставить в стеке 1, перейти по адресу возврата
6	1 6 1 0 0 2 6 2 0 0 3 10	переписать возвращённое значение в y , восстановить верх стека
7	1 1 0 2 6 2 0 0 3 10	взять y и n из стека, перемножить, записать в z в стеке
8	1 1 2 6 2 0 0 3 10	оставить в стеке 2, перейти по адресу возврата
6	2 6 2 0 0 3 10	переписать возвращённое значение в y , восстановить верх стека
7	2 2 0 3 10	взять y и n из стека, перемножить, записать в z в стеке
7	2 2 6 3 10	оставить в стеке 6, перейти по адресу возврата
10	6 10	выдать ответ 6

Грамотный программист, сочиняя программу, всегда представляет себе, в какой примерно машинный код она будет компилироваться — и это помогает писать хорошо работающие программы. При изучении алгоритмов пишется псевдокод, и нужно точно так же держать в уме, как он будет работать на машине с произвольным доступом к памяти.

2.4 Примеры

Итерация и рекурсия, на примере вычисления факториала.

Псевдокод алгоритма 1 переводится в RAM алгоритма 3.

Алгоритм 3 Вычисление факториала итерацией на RAM

Переменные алгоритма 1 размещаются по ячейкам памяти так: $x_1 = n$, $x_2 = m$, $x_3 = i$.

```
1:  $x_2 = 1$ 
2:  $x_3 = 1$ 
3: IF  $x_3 > x_1$  GOTO 7
4:  $x_2 = x_2 * x_3$ 
5:  $x_3 = x_3 + 1$ 
6: GOTO 3
7:  $x_1 = x_2$ 
8: HALT
```

2.5 Сложность алгоритмов

Алгоритм работает за время $t(n)$, если для любых входных данных размера n он останавливается не позднее чем на шаге с номером $t(n)$. Эту функцию $t(n)$ называют *сложностью алгоритма*.

Предполагается, что за шаг выполняется элементарная операция — сложение двух чисел, проверка двух чисел на равенство, и т.п. — и тогда сложность алгоритма определена с точностью до константного множителя.

Определение 1. • $g(n) = O(f(n))$, если $g(n) \leq C \cdot f(n)$ для всех $n \geq n_0$

- $g(n) = o(f(n))$, если $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$.
- $g(n) = \Omega(f(n))$, если $g(n) \geq C \cdot f(n)$ для всех $n \geq n_0$
- $g(n) = \Theta(f(n))$, если $C_1 \cdot f(n) \leq g(n) \leq C_2 \cdot f(n)$ для всех $n \geq n_0$

Например, линейное время работы — это $\Theta(n)$.

Иногда сложность алгоритма оценивается более тонко — подсчитывается количество определённых действий, если его можно подсчитать точно.

Есть ещё один тонкий момент. Машина с произвольным доступом к памяти определена так, что за шаг она может произвести любое арифметическое действие с числами неограниченной величины. Тогда можно «обмануть» определение, записав в числа очень много битов данных, так что одно действие над числами по сути реализует векторную операцию над данными неограниченного размера. Конечно, считать такую операцию за одно действие — это совсем не соответствует смыслу определения. На настоящих компьютерах длина чисел ограничена — скажем, легко выполняются операции над 64-битными числами — и используемая нами математическая модель не должна позволять делать что-то принципиально быстрее, это сделает её неточной моделью.

Как уточнить определение? «Модель логарифмической цены» (log-cost model) — действия над числом i требуют времени $\log i$.

сложность по памяти.



Рис. 1: Анатолий Карацуба (1937–2008).

3 Быстрое умножение, алгоритм Карацубы

Умножение в столбик двух n -разрядных чисел занимает n^2 умножений разряд на разряд — каждая цифра умножается на каждую — и около n^2 сложений. Можно ли быстрее? На заре исследования алгоритмов высказывалась гипотеза, что, очевидно, быстрее нельзя, потому как это просто невозможно — ведь пар цифр же n^2 , и все надо перемножить — да и если б было можно умножать как-то иначе, то это давно бы уже придумали, и поэтому остаётся только разобраться, как подобные очевидные вещи доказывать... А Карацуба построил алгоритм! (см. Карацуба и Офман [1962], а также поздний исторический очерк Карацубы [1995])

Чтобы перемножить два двухразрядных числа, по определению, нужно вычислить 4 произведения и потом сложить их и вычислить перенос.

$$\begin{array}{r}
 \begin{array}{cc}
 a_1 & a_0 \\
 b_1 & b_0
 \end{array} \\
 \hline
 \begin{array}{cc}
 a_1 b_0 & a_0 b_0 \\
 a_1 b_1 & a_0 b_1
 \end{array} \\
 \hline
 \begin{array}{ccc}
 a_1 b_1 & a_1 b_0 + a_0 b_1 & a_0 b_0
 \end{array}
 \end{array}$$

Первая идея метода Карацубы: два двухразрядных числа можно умножить не за 4, а за 3 умножения. Сперва произведения $a_0 b_0$ и $a_1 b_1$ вычисляются как обычно. Затем перемножаются — внезапно — числа $a_1 + a_0$ и $b_1 + b_0$. Выходит $(a_1 + a_0)(b_1 + b_0) = a_1 b_1 + a_0 b_1 + a_1 b_0 + a_0 b_0$, из чего можно вычесть имеющиеся произведения $a_0 b_0$ и $a_1 b_1$, получив желаемую сумму $a_1 b_0 + a_0 b_1$.

Вторая идея метода Карацубы: точно так же можно перемножать два n -разрядных числа, представленных в двоичной записи. Сперва числа представляются в виде $a = a_1 \cdot 2^{\frac{n}{2}} + a_0$, $b = b_1 \cdot 2^{\frac{n}{2}} + b_0$, где $a_0, a_1, b_0, b_1 \in \{0, \dots, 2^{\frac{n}{2}} - 1\}$ — половинки их двоичной записи. Поскольку числа a и b представлены в двоичной системе счисления, чтобы разделить каждое из них таким образом, не надо ни на что делить, а достаточно просто взять $\frac{n}{2}$ младших и $\frac{n}{2}$ старших разрядов их двоичной записи. И дальше всё то же самое, потребуется вычислить 3 произведения чисел длины $\frac{n}{2}$ и $O(n)$ битовых сложений (включая действия с переносом).

Третья идея метода Карацубы: для вычисления каждого из трёх произведений чисел вдвое меньшей длины рекурсивно используется *тот же самый метод*. На каждом уровне

рекурсии размер задачи уменьшается вдвое, и потому глубина рекурсии — $\log_2 n$.

Оценка времени работы алгоритма. Рекурсивные вызовы процедуры умножения образуют дерево. При каждом вызове проделывается какое-то количество расчётов, а также выполняются три рекурсивных вызова. Общее время работы — это сумма времени выполнения расчётов во всех вершинах дерева, сам процесс вызова процедур занимает существенно меньше времени. Подзадача размера n всего одна. Подзадач размера $\frac{n}{2}$ всего 3. Подзадач размера $\frac{n}{2^i}$ всего 3^i . Время работы для каждой подзадачи линейно относительно её размера, и для подзадачи размера $\frac{n}{2^i}$ оно составляет $O(\frac{n}{2^i})$ шагов. Общее число операций подсчитывается так.

$$\sum_{i=0}^{\log_2 n} 3^i \frac{n}{2^i} = n \sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i = n \frac{\left(\frac{3}{2}\right)^{1+\log_2 n} - 1}{\frac{3}{2} - 1} = O(3^{\log_2 n}) = O(n^{\log_2 3})$$

Основные идеи этого алгоритма впоследствии многократно применялись при построении алгоритмов для решения разнообразных задач. Общий метод получил название **метода «разделяй и властвуй»**. Согласно этому методу, задача разделяется на непересекающиеся подзадачи того же типа, но меньшего размера, каждая подзадача решается отдельно, а потом результаты решения объединяются в решение исходной задачи.

4 Алгоритмы сортировки

Задача: дан массив $a_1, \dots, a_n$ из целых чисел; нужно поменять их местами, применив некоторую перестановку $(i_1, \dots, i_n)$, чтобы выполнялось $a_{i_1} \leq \dots \leq a_{i_n}$.

Более общая формулировка: пусть X — множество возможных значений элементов с заданным на нём транзитивным и полным отношением сравнения ($\leq$): для всяких $x, y \in X$ выполняется $x \leq y$ или $x \geq y$, или и то и другое одновременно (последний случай обозначается через $x \equiv y$); если $x, y, z \in X$, $x \leq y$ и $y \leq z$, то $x \leq z$. Задача: дан массив $a_1, \dots, a_n$ из элементов X , нужно поменять их местами так, чтобы выполнялось $a_{i_1} \leq \dots \leq a_{i_n}$.

Алгоритм сортировки — *устойчивый* (stable), если элементы с одинаковым значением сохраняют свой порядок, то есть, если в исходном массиве выполнялось $a_i \equiv a_j$, где $i < j$, то в отсортированном массиве a_i будет расположен раньше, чем a_j .

4.1 Сортировка вставкой

Самый очевидный алгоритм сортировки — *сортировка вставкой* (insertion sort). Главная мысль: сперва отсортировать все элементы с 1-го по $(i-1)$ -й, потом вставить i -й элемент на своё место среди элементов с 1-го по $(i-1)$ -й. Для этого нужно сдвинуть на одну позицию вперёд все элементы между правильным местом и i -м местом. И так далее, для всех i от 2 до n .

Доказательство правильности алгоритма (что на таких-то входных данных он действительно останавливается и выдаёт такой-то результат).

Лемма 1. Алгоритм 4 правильно сортирует полученный на входе массив.

Доказательство. Инвариант внешнего цикла: перед итерацией i , текущие значения $a_1, \dots, a_{i-1}$ содержат отсортированную последовательность $a_1, \dots, a_{i-1}$ элементов первоначального массива; после итерации i , элементы $a_1, \dots, a_i$ отсортированы.

Алгоритм 4 Сортировка вставкой (insertion sort)

```
1: for  $i = 2$  to  $n$  do
2:    $t = a_i$ 
3:    $j = i - 1$ 
4:   while  $j \geq 1$  и  $a_j > t$  do
5:      $a_{j+1} = a_j$ 
6:      $j = j - 1$ 
7:    $a_{j+1} = t$ 
```

Доказывается индукцией по i .

Для $i = n$ — то есть, по окончании работы алгоритма — стало быть, отсортировано всё. $\square$

Оценка времени работы алгоритма в худшем случае. На данном входе w размера n — время работы $T(w)$. На всех входах размера n полагается $T(n) = \max_{|w|=n} T(w)$.

Для сортировки вставкой время работы: $\Theta(n^2)$. На удачно подобранных (почти отсортированных) входных данных может работать быстрее.

4.2 Сортировка слиянием

Рекурсивный алгоритм, основанный на методе «разделяй и властвуй». Каждый вызов $merge_sort(i, j)$ сортирует участок массива $a_i, \dots, a_{j-1}$. Если $j - i = 1$, то одноэлементный участок уже отсортирован. Если $j - i \geq 2$, то участок делится на два куска равного размера, каждый кусок рекурсивно сортируется, после чего два отсортированных куска *сливаются* в один отсортированный кусок с помощью процедуры $merge()$.

Алгоритм 5 Сортировка слиянием (merge sort) (фон Нейман).

Дано: массив $a_1, \dots, a_n$ из целых чисел. Поменять их местами так, чтобы выполнялось $a_1 \leq \dots \leq a_n$.

Вызывается процедура $merge_sort(1, n + 1)$.

Процедура $merge_sort(\ell, m)$

```
1: if  $m - \ell \geq 2$  then
2:    $merge\_sort(\ell, \lfloor \frac{\ell+m}{2} \rfloor)$  /* отсортировать первую половину */
3:    $merge\_sort(\lfloor \frac{\ell+m}{2} \rfloor, m)$  /* отсортировать вторую половину */
4:    $merge(\ell, \lfloor \frac{\ell+m}{2} \rfloor, m)$  /* слить две отсортированные половины */
```

Процедура $merge(\ell, p, m)$

```
1:  $b[1..p - \ell] = a[\ell..p - 1]$  /* скопировать первую половину в массив  $b$  */
2:  $c[1..m - p] = a[p..m - 1]$  /* скопировать вторую половину в массив  $c$  */
3:  $i = 1$  /* указатель на наименьший элемент массива  $b$  */
4:  $j = 1$  /* указатель на наименьший элемент массива  $c$  */
5: for  $t = \ell$  to  $m - 1$  do
6:   if  $j = m - p \vee (i < p - \ell \wedge b[i] \leq c[j])$  then
7:      $a[t] = b[i]$  /* взять следующий элемент из  $b$  */
8:      $i = i + 1$  /* переместить указатель на элемент  $b$  */
9:   else
10:     $a[t] = c[j]$  /* взять следующий элемент из  $c$  */
11:     $j = j + 1$  /* переместить указатель на элемент  $c$  */
```

Процедура $merge(\ell, p, m)$ работает за время $m - \ell$.

Оценка времени работы всего алгоритма для n — степени двойки. Глубина рекурсии — $\log_2 n$. Задач размера n — одна. Задач размера $\frac{n}{2}$ — две. Задач размера $\frac{n}{2^i}$ всего 2^i . Время работы $merge$ для каждой задачи размера $\frac{n}{2^i}$ составляет $O(\frac{n}{2^i})$ шагов. Всего:

$$\sum_{i=0}^{\log_2 n} 2^i \frac{n}{2^i} = n \log_2 n$$

Список литературы

- [1936] A. Church, “An unsolvable problem of elementary number theory”, *American Journal of Mathematics*, 58 (1936), 345–363.
- [1995] А. А. Карацуба, “Сложность вычислений”, *Труды Математического института РАН*, 211 (1995), 186–202.
- [1962] А. А. Карацуба, Ю. П. Офман, “Умножение многозначных чисел на автоматах”, *Доклады Академии Наук СССР*, 145:2 (1962), 293–294.
- [1937] A. M. Turing, “On computable numbers, with an application to the Entscheidungsproblem”, *Proceedings of the London Mathematical Society, Series 2*, 42:1 (1937), 230–265.